

CURRENT SYSTEM DOCUMENTATION

## KPTC Scheduler

# はじめての独立Linuxサーバー構築・移行手順書

内部スケジューラーと外部空き状況ページを、別々のUbuntu Serverへ安全に構築する実作業ガイド

|                   |              |                   |
|-------------------|--------------|-------------------|
| 2台<br>内部 / 外部サーバー | 20<br>作業ステップ | 1方向<br>内部から外部への送信 |
|-------------------|--------------|-------------------|

この資料の基準  
文書版 v4.0 / 更新日 2026-08-30 / 実装基準コミット 6920f43 / ブランチ main

# 01 最初に読む

この手順書は、専門家でない方でも、作業場所・実行コマンド・成功条件を順番に確認できるよう構成しています。コピー後は必ず「正常なら」の結果を確認してから次へ進んでください。

| 対象     | 固定する条件                                    | 完成後                     |
|--------|-------------------------------------------|-------------------------|
| 内部サーバー | Ubuntu Server 24.04 LTS / Nginx / PHP-FPM | 社内LAN・VPN・社内システムから予定を利用 |
| 外部サーバー | Ubuntu Server 24.04 LTS / Nginx / PHP-FPM | インターネットから3室の空き状況を閲覧     |
| データ連携  | 内部から外部への署名付きHTTPSのみ                       | 外部に内部DB・個人情報を置かない       |

管理担当者へ先に確認

内部・外部FQDN、内部アクセス可能範囲、外部公開可否、TLS証明書、社内システムからカレンダーへ到達させる方法を確定してください。

## 02 開始条件と中止条件

| 確認        | 必要な状態                     | 判定 |
|-----------|---------------------------|----|
| サーバー      | 内部用・外部用のUbuntu Serverが各1台 | □  |
| 管理権限      | 両サーバーへSSH接続しsudoを実行可能     | □  |
| 内部接続      | 職員PCから内部443番へ接続可能         | □  |
| 外部接続      | 外部443番をインターネットへ公開可能       | □  |
| DNS / TLS | 2つのFQDNと証明書方針が決定          | □  |
| 時刻同期      | 両サーバーで有効、ずれ5分未満           | □  |
| バックアップ    | 内部DBと設定の保存先が決定            | □  |

作業を停止

SSH切断、sudo不可、空き容量2GB未満、既存Webへの影響が不明、正式バックアップがない場合は先へ進まないください。

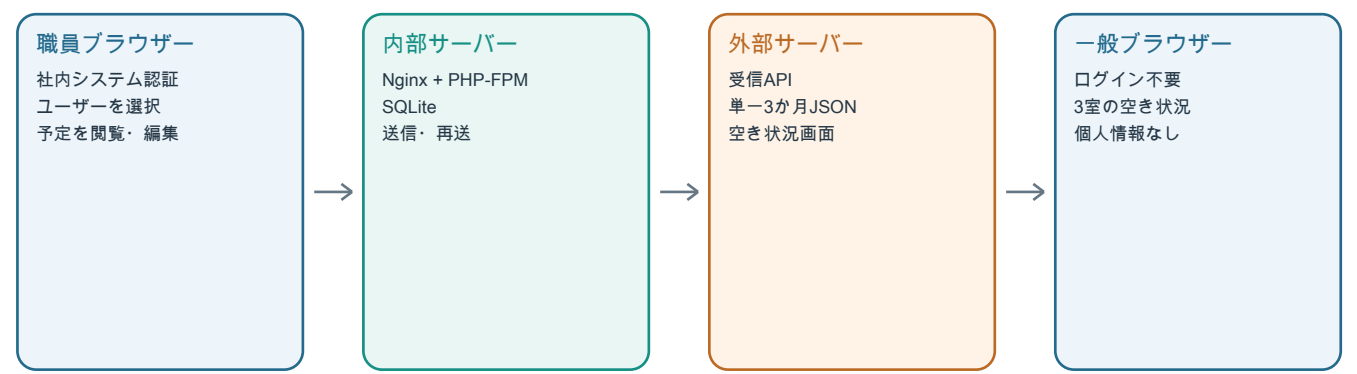
### 03 記入シート

| 項目            | 記入欄                                                 | 例                          |
|---------------|-----------------------------------------------------|----------------------------|
| 内部SSH接続先      | _____                                               | admin@10.20.1.10           |
| 外部SSH接続先      | _____                                               | admin@203.0.113.20         |
| 内部FQDN        | _____                                               | scheduler.intra.example.jp |
| 外部FQDN        | _____                                               | availability.example.jp    |
| 内部許可範囲        | _____                                               | 10.20.0.0/16               |
| 初期管理者ID       | _____                                               | admin                      |
| 紐付けるmember_id | _____                                               | m1                         |
| Gitコミット       | 6920f43                                             | この版の基準                     |
| 内部DB          | /var/lib/kptc-scheduler/group-watcher.sqlite        | 推奨値                        |
| 外部JSON        | /var/lib/kptc-availability/public-availability.json | 推奨値                        |

秘密値

共有秘密鍵はこのPDFや作業メモへ書かず、組織の秘密管理場所へ保存します。GitHub・メール・チャットへ貼り付けません。

# 04 完成構成と作業順



| 到達点      | 完了の目印                          |
|----------|--------------------------------|
| A ソース準備  | 指定コミットを取得し、check/test/buildが成功 |
| B 内部基盤   | NginxとPHP-FPMがactive           |
| C 内部画面   | ユーザー選択画面をHTTPSで表示              |
| D 外部画面   | 空き状況画面をHTTPSで表示                |
| E JSON連携 | 内部から外部へ手動送信成功                  |
| F 自動再送   | publish / monitor timerがactive |
| G アクセス制限 | 社内システムを経由しない内部URLを拒否           |
| H 受入     | バックアップ・監視・操作試験が全て合格            |

## 05 作業用PCへソースを取得

実行場所: 作業用PCのターミナル

```
cd ~/Documents
git clone https://github.com/Apfelengineer/260725_GW_WEB.git
cd 260725_GW_WEB
git checkout 6920f43
git log -1 --oneline
```

正常なら

git logの先頭が 6920f43 です。既存の同名フォルダーを削除せず、新しい空フォルダーで実施してください。

### server-runtime-snapshotについて

このフォルダーは過去に取得したサーバー実行ファイルの参照用です。現在版の配布元には使用せず、必ずこの手順でソースからdist-internal / dist-publicを再生成します。

## 06 Node.js・pnpmを準備してビルド

Node.js 22.13以上を使用します。組織指定の導入方法がある場合はそちらを優先してください。

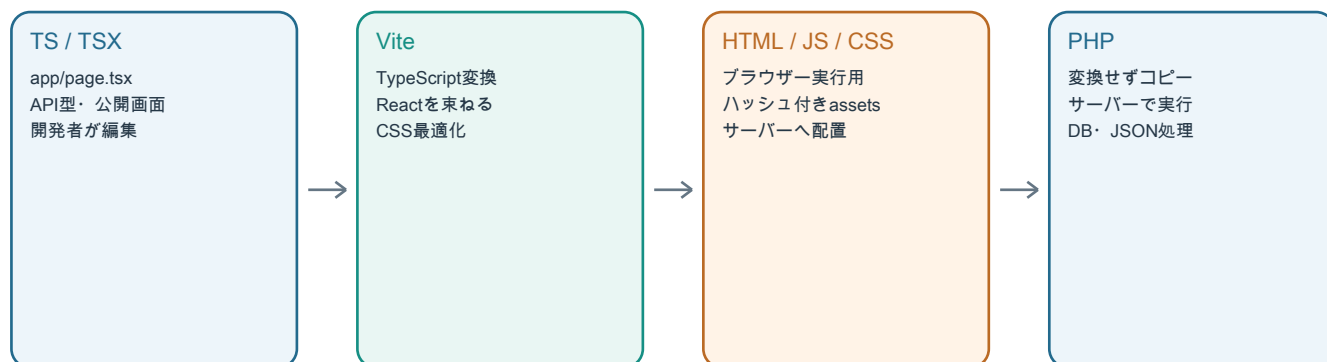
```
node --version
corepack enable
corepack prepare pnpm@latest --activate
pnpm --version
pnpm install --frozen-lockfile
pnpm run check
pnpm test
pnpm run build
```

正常なら

TypeScriptエラー0、テスト8件pass、dist-internalとdist-publicが生成されます。

| 生成先           | ファイル数 | 用途                        |
|---------------|-------|---------------------------|
| dist-internal | 13    | 内部スケジューラー画面、API、認証・送信CLI  |
| dist-public   | 10    | 外部空き状況画面、受信・公開・health API |

## 07 ビルド後ファイルを理解する



### 重要

サーバーには.tsxを置きません。内部サーバーへdist-internalの中身、外部サーバーへdist-publicの中身を配置します。SQLite・設定・JSONは配布物とは別に保持します。



## 08 両サーバーへ基本ソフトを導入

実行場所: 内部サーバー、続いて外部サーバー

```
sudo apt update
sudo apt install -y nginx php-fpm php-cli php-sqlite3 php-curl php-mbstring unzip
sudo systemctl enable --now nginx php8.3-fpm
systemctl is-active nginx
systemctl is-active php8.3-fpm
php -v
php -m | grep -E 'curl|mbstring|pdo_sqlite'
```

正常なら

2つのsystemctl結果がactiveで、PHPモジュール3つが表示されます。

## 09 内部サーバーのフォルダーを作成

```
sudo mkdir -p /opt/kptc-scheduler/internal/releases/20260830-6920f43
sudo mkdir -p /var/lib/kptc-scheduler
sudo mkdir -p /etc/kptc-scheduler
sudo chown -R www-data:www-data /var/lib/kptc-scheduler
sudo chmod 750 /var/lib/kptc-scheduler
sudo chmod 750 /etc/kptc-scheduler
```

作業用PCから送信:

```
rsync -av dist-internal/ INTERNAL_SSH:/tmp/kptc-internal/
```

内部サーバーで配置:

```
sudo rsync -a /tmp/kptc-internal/ /opt/kptc-scheduler/internal/releases/20260830-6920f43/
sudo ln -sf /opt/kptc-scheduler/internal/releases/20260830-6920f43 /opt/kptc-scheduler/internal/current
sudo chown -R root:www-data /opt/kptc-scheduler/internal/releases/20260830-6920f43
sudo find /opt/kptc-scheduler/internal/releases/20260830-6920f43 -type d -exec chmod 750 {} \;
sudo find /opt/kptc-scheduler/internal/releases/20260830-6920f43 -type f -exec chmod 640 {} \;
```

## 10 内部サーバーの設定ファイル

`/etc/kptc-scheduler/internal-server.env`をroot権限で作成し、次の値を実環境へ置換します。

```
KPTC_INTERNAL_SCHEDULER_DB=/var/lib/kptc-scheduler/group-watcher.sqlite
KPTC_PUBLIC_AVAILABILITY_MODE=https
KPTC_PUBLIC_AVAILABILITY_ENDPOINT=https://availability.example.jp/receive-availability.php
KPTC_PUBLIC_AVAILABILITY_PAGE_URL=https://availability.example.jp/calendar/
KPTC_PUBLIC_AVAILABILITY_SECRET=REPLACE_LATER
KPTC_PUBLIC_AVAILABILITY_TIMEOUT=10
KPTC_PUBLIC_AVAILABILITY_STALE_SECONDS=1800
KPTC_SESSION_COOKIE_SECURE=1
KPTC_AUTH_IDLE_TIMEOUT=1800
KPTC_AUTH_ABSOLUTE_TIMEOUT=43200

sudo chown root:www-data /etc/kptc-scheduler/internal-server.env
sudo chmod 640 /etc/kptc-scheduler/internal-server.env
```

### Web用PHPへの設定

Nginx/PHP-FPMでは、専用poolのenv[]設定、またはWeb公開領域外のinternal-env.phpを使用します。組織のPHP運用規約に合わせ、CLIとWebで同じ値になることを確認してください。

## 11 内部NginxとPHP-FPM

内部FQDN用のserverブロックを作成します。次は要点です。

```
server {
    listen 80;
    server_name scheduler.intra.example.jp;
    root /opt/kptc-scheduler/internal/current;
    index index.html;

    location / { try_files $uri $uri/ /index.html; }
    location ~ /\.php$ {
        include snippets/fastcgi-php.conf;
        fastcgi_pass unix:/run/php/php8.3-fpm.sock;
    }
}

sudo nginx -t
sudo systemctl reload nginx
curl -I http://scheduler.intra.example.jp/
curl 'http://scheduler.intra.example.jp/api.php?action=bootstrap'
```

正常なら

画面はHTTP 200、bootstrapはJSONを返します。PHPソースが文字列として表示された場合は直ちに公開を止め、PHP-FPM設定を修正してください。

## 12 初期管理者を作成

初回bootstrapでSQLiteと初期メンバーを作成した後、管理者を作成します。パスワードは不要です。

```
sudo -u www-data env KPTC_INTERNAL_SCHEDULER_DB=/var/lib/kptc-scheduler/group-watcher.sqlite php /opt/kptc-scheduler/internal/cur  
sudo -u www-data env KPTC_INTERNAL_SCHEDULER_DB=/var/lib/kptc-scheduler/group-watcher.sqlite php /opt/kptc-scheduler/internal/cu
```

正常なら

Created: adminと表示され、listにusername=admin、member\_id=m1、role=adminが含まれます。

注意

m1は新規DBの初期メンバー例です。既存DBを移行する場合は、実際の管理者へ紐づくmember\_idを確認して指定します。

## 13 外部サーバーを配置

```
sudo mkdir -p /opt/kptc-scheduler/public/releases/20260830-6920f43
sudo mkdir -p /var/lib/kptc-availability
sudo mkdir -p /etc/kptc-availability
sudo chown -R www-data:www-data /var/lib/kptc-availability
sudo chmod 750 /var/lib/kptc-availability /etc/kptc-availability
```

作業用PCから送信:

```
rsync -av dist-public/ EXTERNAL_SSH:/tmp/kptc-public/
```

外部サーバーで配置:

```
sudo rsync -a /tmp/kptc-public/ /opt/kptc-scheduler/public/releases/20260830-6920f43/
sudo ln -sf /opt/kptc-scheduler/public/releases/20260830-6920f43 /opt/kptc-scheduler/public/current
sudo chown -R root:www-data /opt/kptc-scheduler/public/releases/20260830-6920f43
```

## 14 外部設定・Nginx

`/etc/kptc-availability/external-server.env`:

```
KPTC_PUBLIC_AVAILABILITY_JSON=/var/lib/kptc-availability/public-availability.json
KPTC_PUBLIC_AVAILABILITY_SECRET=REPLACE_LATER
KPTC_PUBLIC_AVAILABILITY_STALE_SECONDS=1800
```

外部Nginxの要点:

```
server {
    listen 80;
    server_name availability.example.jp;
    root /opt/kptc-scheduler/public/current;
    index index.html;
    location / { try_files $uri $uri/ /index.html; }
    location ~ /\.php$ {
        include snippets/fastcgi-php.conf;
        fastcgi_pass unix:/run/php/php8.3-fpm.sock;
    }
}

sudo nginx -t
sudo systemctl reload nginx
curl -I http://availability.example.jp/
curl -i http://availability.example.jp/health-availability.php
```

初回health=503

まだJSONを受信していないため正常です。第16章の送信後に200へ変わります。

## 15 HTTPSを有効化

組織指定の証明書がある場合はその手順を優先します。Let's Encryptが許可される環境の例:

```
sudo apt install -y certbot python3-certbot-nginx
sudo certbot --nginx -d scheduler.intra.example.jp
sudo certbot --nginx -d availability.example.jp
sudo certbot renew --dry-run
```

正常なら

ブラウザーに証明書警告がなく、両URLがhttps://で表示されます。HTTPはHTTPSへ転送します。



## 16 共有秘密鍵と手動送信

内部サーバーで秘密鍵を生成します。表示された値を秘密管理場所へ保存し、内部・外部の設定へ同じ値を1回ずつ設定します。

```
openssl rand -hex 32
```

設定反映後、PHP-FPMを再起動し、内部サーバーで手動送信:

```
sudo systemctl restart php8.3-fpm
set -a
. /etc/kptc-scheduler/internal-server.env
set +a
sudo -E -u www-data php /opt/kptc-scheduler/internal/current/publish-availability-cli.php
unset KPTC_PUBLIC_AVAILABILITY_SECRET

curl -i https://availability.example.jp/health-availability.php
```

正常なら

送信CLIは成功JSONと終了コード0、外部healthはHTTP

200を返します。シェル履歴へ秘密値を直接入力しない運用へ切り替えてください。

## 17 自動再送・監視を有効化

```
sudo cp deploy/kptc-availability-publish.service /etc/systemd/system/  
sudo cp deploy/kptc-availability-publish.timer /etc/systemd/system/  
sudo cp deploy/kptc-availability-monitor.service /etc/systemd/system/  
sudo cp deploy/kptc-availability-monitor.timer /etc/systemd/system/  
sudo systemctl daemon-reload  
sudo systemctl enable --now kptc-availability-publish.timer  
sudo systemctl enable --now kptc-availability-monitor.timer  
systemctl list-timers | grep kptc
```

正常なら

publishは5分間隔、monitorは10分間隔で次回実行時刻が表示されます。

障害確認:

```
systemctl status kptc-availability-publish.timer  
journalctl -u kptc-availability-publish.service -n 50 --no-pager  
journalctl -u kptc-availability-monitor.service -n 50 --no-pager
```

# 18 社内システム認証の内側へ置く

最重要

KPTC Schedulerはパスワードを確認せず、選択したユーザーの権限を付与します。内部URLを直接開けると、誰でも管理者を選べます。実データ投入前に必ずアクセス制限を完成させます。

実装方法は組織基盤に合わせ、次のいずれかを採用します。

- 社内ポータルのリバースプロキシ配下に置き、未認証要求をポータルのログインへ転送。
- OIDC/SAML対応プロキシで認証済み要求だけを内部Nginxへ転送。
- VPNまたは社内LANからのみ443番を許可し、外部経路をファイアウォールで遮断。
- 上位システムからiframe表示する場合も、内部URL自体の直接アクセスを拒否。

## 確認方法

| 試験           | 合格条件                   |
|--------------|------------------------|
| 社内システム未ログイン  | 内部URLを開けない / 上位ログインへ転送 |
| 社内システムログイン済み | ユーザー選択画面を表示            |
| 社外ネットワーク     | DNS非解決または接続拒否          |
| 外部空き状況       | 社外からHTTPSで表示可能         |

## 19 既存DB移行・バックアップ・復旧

停止時間を確保し、旧環境で書き込みを止めてからSQLiteをコピーします。

```
# ■■■
sqlite3 group-watcher.sqlite 'PRAGMA integrity_check;'
cp -p group-watcher.sqlite group-watcher.sqlite.pre-migration

# ■■■■■■■■
sudo systemctl stop php8.3-fpm
sudo cp group-watcher.sqlite /var/lib/kptc-scheduler/group-watcher.sqlite
sudo chown www-data:www-data /var/lib/kptc-scheduler/group-watcher.sqlite
sudo chmod 640 /var/lib/kptc-scheduler/group-watcher.sqlite
sudo systemctl start php8.3-fpm
```

- DBと設定は毎日バックアップし、世代・保存期間・復元試験日を台帳化。
- アプリ更新は新releaseへ配置し、currentシンボリックリンクを切替。
- 不具合時はcurrentを直前releaseへ戻し、DBは必要時だけ停止後に復元。
- 公開JSONは内部予定から再生成できるため、履歴バックアップは必須ではない。

## 20 最終受入チェック

| 確認     | 合格条件                        | 判定                       |
|--------|-----------------------------|--------------------------|
| 内部画面   | HTTPS、社内経路のみ、ユーザー選択・ゲスト表示   | <input type="checkbox"/> |
| 権限     | admin/user/room/guestが仕様どおり | <input type="checkbox"/> |
| 予定操作   | 登録・編集・コピー・削除・複数選択           | <input type="checkbox"/> |
| ドラッグ   | 確認画面に移動元/先、キャンセルは未変更        | <input type="checkbox"/> |
| 外部画面   | 3室、直近3か月、土日祝、記号・濃灰          | <input type="checkbox"/> |
| 連携     | 手動送信成功、health=200、個人情報なし    | <input type="checkbox"/> |
| 再送     | 2つのtimer active、ログに継続エラーなし  | <input type="checkbox"/> |
| バックアップ | DB・設定を別媒体へ保存し復元確認           | <input type="checkbox"/> |
| 資料     | 実URL・担当者・緊急停止手順を運用台帳へ記録     | <input type="checkbox"/> |

完了

全項目が合格し、社内システム未認証時の直接アクセスが遮断された時点で本番利用を開始します。